ROSEBANK
INTERNATIONAL

ANDISWA NONDUDUZO MCHUNU

FACULTY OF ICT
ADVANCED DIPLOMA ICT
APPLICATION DEVELOPMENT
PROGRAMMING 3A
ENTERPRISE APPLICATION
DEVELOPMENT

TITLE

PortFolio of Evidence

TECHNICAL REFLECTION REPORT

1. Introduction

So, for my project, I built the Global Logistics Management System (GLMS)-an enterprise app designed for TechMove Logistics to handle clients, service contracts, service requests, signed agreements, and currency conversions.

Basically, I modernized GLMS by moving away from a traditional ASP.NET Core MVC app and turning it into a service-oriented architecture. Now, it uses a separate ASP.NET Core Web API, shared domain models, and a SQL Server database.

Separating the presentation stuff from the core business logic makes it way easier to test and deploy parts of it independently. I also messed around with some DevOps stuff, like automated testing in CI/CD pipelines and Docker containerisation. This report is basically my breakdown of how these practices make life easier, cut down on deployment headaches, and stop the classic "it works on my machine" problem.

2. DevOps and Automated Testing

The Importance of Automated Testing in CI/CD

Automated testing is super clutch in modern software development because it checks your code automatically every time you make a change.

With Continuous Integration, developers are constantly pushing updates to a shared repository. According to Microsoft (n.d.-a), GitHub Actions can automatically run .NET builds and tests whenever you push new code, giving you instant feedback on whether your app is still working.

Without automated testing, you're basically flying blind and hoping you didn't break anything. Automated testing catches those slip-ups early. You can set tests to trigger automatically whenever someone pushes code or opens a pull request.

For GLMS, this is a big deal because the system handles sensitive info like client records, service agreements, and contract statuses.

How Automated Testing Prevents Bugs from Production

Automated testing stops bugs from sneaking into production in a few key ways:

- **Early defect detection:** Tests catch weird behavior right after you write the code, making bugs way easier to debug before they snowball.
- **Regression testing:** Tests run your old features over and over to make sure new code didn't accidentally break something that was already working.
- **Integration testing:** Microsoft (n.d.-b) notes that ASP.NET Core integration testing checks how different pieces talk to each other-like your app interacting with databases, file systems, and network calls.
- **Continuous feedback:** You immediately know if your code actually meets the requirements.
- **Deployment quality gates:** You can set up your CI/CD pipeline to flat-out block code from deploying if the tests fail.

OfC, automated testing doesn't magically mean your code is 100% bug-free, but it drastically lowers the odds of breaking things in production.

Automated Testing in GLMS

In GLMS, my testing strategy covers both basic business logic and how components interact.

- **Unit tests** are great for checking isolated rules, like making sure contract dates make sense, checking status transitions, validating files, and running currency math.
- **Integration tests** are crucial for verifying how the MVC frontend, Web API, and data layer talk to each other.

For instance, an integration test can send a mock contract creation request straight to an API endpoint and check if the app spits back the correct HTTP response.

As Microsoft (n.d.-b) explains, you can use `WebApplicationFactory` to spin up an ASP.NET Core app in a test environment and generate HTTP clients to test responses.

My GLMS solution uses an xUnit testing project as the base for this. The main takeaway here is that just because your code compiles doesn't mean it actually works the way you want it to- automated testing gives you that peace of mind.

3. Docker Containerisation

Environment Consistency and the "It Works on My Machine" Problem

We've all been there: your code runs completely fine on your local machine, but the second you push it to a test server or production, it completely crashes.

Usually, that happens because of annoying differences in operating systems, runtime versions, missing dependencies, or weird environment variables.

Docker fixes most of that by packing your app and everything it needs into a container image. Running the same container image across different environments means you don't have to stress about missing software. That said, you still have to manage your environment-specific configs and external services properly.

How Docker Improves Consistency

Docker images give you a reliable, repeatable way to run your app.

A Dockerfile tells Docker step-by-step how to build your image-what base environment to use, what dependencies to pull in, how to compile, and how to start up.

Docker (n.d.-a) points out that multi-stage builds are awesome because they separate your compilation phase from the actual runtime environment, meaning you can strip out heavy development tools and keep your production images lean and small.

For GLMS, I set up separate Dockerfiles for the ASP.NET Core MVC frontend and the Web API. The build stage uses the heavy .NET SDK, while the runtime stage uses the much lighter ASP.NET Core runtime image.

Docker Compose Architecture for GLMS

For the GLMS deployment, I worked with a three-tier setup:

- **MVC frontend:** Handles the user interface and talks to the backend via HTTP requests.
- **Web API:** Processes requests, runs the business logic, and talks to the database.
- **SQL Server:** Stores all the client info, contracts, and service requests.

Docker Compose makes managing this a breeze using a single YAML configuration file to link everything together (Docker, n.d.-b).

In GLMS, the `docker-compose.yml` file configures the frontend, backend, and database to run as a multi-container app on a shared Docker network, so you don't have to manually set up your local dev environment from scratch.

Persistent Storage

When you're containerising a database app, data persistence is super important.

Containers are meant to be disposable-they get recreated during deployments, updates, or troubleshooting. If you store data inside the container's temporary writable layer, it gets wiped out the second the container stops.

Docker (n.d.-c) recommends using volumes for persistent data since their lifecycle is tied to the host system, not the container.

For GLMS, this is critical for keeping our SQL Server records and uploaded signed agreement files safe.

Limitations of Docker

Even though Docker makes environments way more consistent, it's not a silver bullet.

Your app can still break if you mess up connection strings, forget to set an environment variable, run into network issues, or try to reach an external service that's down.

For example, the SQL Server LocalDB connection string you use locally in Visual Studio won't magically work inside a SQL Server container. You still have to pay attention to your configurations.

4. Architectural Modernisation of GLMS

Separation of Concerns

The new GLMS architecture cleanly splits up the presentation layer, the API layer, business services, and database operations.

The request flow looks like this:

MVC Frontend → HttpClient → Web API → Business Services → Entity Framework Core → SQL Server

By putting shared domain models into a separate class library, I cut down a ton of code duplication. This makes maintenance way easier because you can update the UI without accidentally breaking your backend logic, and you can test the backend entirely on its own.

Dependency Injection

Using dependency injection keeps all our components nicely decoupled.

In GLMS, I used service interfaces like IContractService, IServiceRequestService, and ICurrencyService to map out the business operations, registering them right into the ASP.NET Core DI container.

This makes testing super straightforward because you can easily swap out real services for mock implementations during unit tests.

Currency Conversion and Resilience

For the currency conversion feature, I implemented the Strategy design pattern.

There's a live strategy that grabs fresh exchange rates from an external HTTP API, and a cached strategy that acts as a fallback if the live service goes down.

It's a great example of how you can build resilience into an app, and it makes testing way easier since you can simulate API outages without breaking your tests.

5. Technical Reflection and Lessons Learned

Managing Dependencies

Breaking up the original monolith MVC app into multiple smaller projects taught me real quick how important it is to manage dependencies carefully.

Moving business services and models around meant fixing a ton of broken namespaces, project references, and DI registrations. It definitely showed me how painful it is when code is too tightly coupled, and made me appreciate modular design way more.

Testing Beyond Compilation

Getting your code to compile successfully just means the syntax is right-it doesn't mean your services actually start up or talk to each other correctly.

That's why integration testing is so huge. As Microsoft (n.d.-b) highlights, integration tests check how components actually behave with their real infrastructure, like testing actual API calls, database saves, and service execution all at once.

Deployment Reproducibility

Using Docker proved how nice it is to have reproducible environments.

Instead of forcing every developer to manually install the exact right SDKs and tools on their personal laptops, Dockerfiles and Docker Compose let us spin up the exact same environment with a single command.

The Importance of DevOps

Working on GLMS really drove home the point that software engineering isn't just about writing code that works on your machine.

Automated testing, version control, clean modular architecture, and containerisation all work together to make sure software is actually reliable when it ships.

6. Conclusion

Modernising GLMS showed me firsthand why DevOps practices are so crucial for enterprise development.

Automated testing catches bugs early and stops regressions, Docker keeps your environments consistent, and splitting the app into an MVC frontend, Web API, and shared models makes the codebase way easier to maintain and test.

Overall, putting all these pieces together makes enterprise software a lot more robust and portable.

7. References

Docker (n.d.-a) *Multi-stage builds*. Available at: https://docs.docker.com/build/building/multi-stage/ (Accessed: 11 November 2026).

Docker (n.d.-b) *How Compose works*. Available at: https://docs.docker.com/compose/intro/compose-application-model/ (Accessed: 11 November 2026).

Docker (n.d.-c) *Volumes*. Available at: https://docs.docker.com/engine/storage/volumes/ (Accessed: 11 November 2026).

Microsoft (n.d.-a) *Create a test validation GitHub workflow*. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/devops/dotnet-test-github-action (Accessed: 11 November 2026).

Microsoft (n.d.-b) *Integration tests in ASP.NET Core*. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/aspnet/core/testing/integration-testing (Accessed: 11 November 2026).

8. Project Repository

andiswamchunu/GLMS